

In [574... *#Week 3 Critical Thinking Option 1 - Boston Housing Simple Linear Regression*
#John Wensink
#CSC525 Principles of Machine Learning
#Colorado State University - Global Campus
#Dr. Dong Nguyen
#May 30, 2025

In [575... *#In this project we'll implement a simple linear regression model to the Boston Housing dataset to predict housing prices based on various predictors. We'll explore the dataset, evaluate the model and create some visualizations for feature analysis to see what the leading factors for median home prices were in 1978 Boston*

```
from sklearn.linear_model import LinearRegression as lr
 #(Scikit-Learn Developers, 2024a)
```

#We'll use the basic LinearRegression model from sklearn's linear_model module to fit and predict linear relationships in our dataset. This model is a basic implementation of ordinary least squares regression. The model seeks to minimize the residual sum of squares between observed targets and predicted values.

#Sklearn's linear_model library has a ton of different regression models ranging from the classics like Linear Regression (LR), Ridge, Lasso, and Elastic Net that address issues like overfitting and feature selection. Bayesian linear models (Bayesian Ridge, ARD Regression) that allow for some uncertainty estimation in the prediction of model parameters by placing probabilistic priors on coefficients. Stochastic models like SGD Regressor that use stochastic gradient descent to fit linear models incrementally for large-scale datasets allowing for flexibility and the use of different loss functions/regularization techniques when scalability and efficiency are important. Then there are generalized linear models like Poisson Regressor, Gamma Regressor, and Tweedie Regressor that extend linear modeling to handle non-normal target distributions. These models can be useful when the assumptions of ordinary least squares don't make sense, and offer more appropriate error structures for link functions and other specialized tasks.

#For our purposes today, we'll start with basic LR. This makes sense for this dataset because the relationship between predictors and the target is likely to be (approximately) linear, the dataset is already clean and structured, and our interest is understanding how different features will influence the outcome. LR should provide an easily interpretable baseline model that will allow us to quickly identify trends, evaluate the predictive value of the model, and visualize the distribution of each feature through its coefficients.

In [576... *#Let's start out with a fake dataset to simulate a simple linear relationship...*
y=2x+1
#In sklearn X must be a 2D structure so we'll make a list of lists
X = [[1], [2], [3], [4], [5]]
#Regression is predicting a single target value for each input sample, so Y can be a 1D array
y = [3, 5, 7, 9, 11]
#Let's train the model now to fit the data
model = lr()
model.fit(X, y)
#Let's see if the model shows that it has learned that the relationship is y=2x+1

```
print("Slope (coef_):", model.coef_)
print("Intercept (intercept_):", model.intercept_)
```

#We begin by simulating a simple linear relationship using the most basic dataset imaginable where the underlying function is known to be $y = 2X + 1$. In sklearn, input features (X) need to be in a 2D structure (i.e. a feature matrix), so we format our values as a list of lists. The corresponding target values (y) follow the defined linear pattern and are fine as a 1D structure (i.e. a target vector.) When we instantiate and train the LR model, after fitting, we can inspect the learned slope (coef_) and intercept (intercept_) to confirm that the model has correctly identified the relationship, which suggests that our model is operating as intended.

#It's interesting to point out that standard mathematical notation, particularly in linear algebra and machine learning makes use of capitalized and lower case variables to distinguish between different types of data structures. We see here that the capital (X) is representing a 2D matrix, where each column is a feature and each row is a datapoint. In contrast, the lowercase (y) denotes a vector, a 1D array of target values corresponding to each row in (X.) This convention helps to quickly convey the role and dimensionality of variables within predictive models like LR (Mahmood, 2024.)

```
Slope (coef_): [2.]
Intercept (intercept_): 1.0
```

```
In [577... #Now Let's see if the model will predict a new value
X_test = [[6], [7], [8], [9]]
predictions = model.predict(X_test)
print("Predicted values = ", predictions)
```

#After training our regression model on the initial dataset, here we tested its ability to generalize by providing new, unseen input values. Again (X_test) is given as a list of lists. We use the model's .predict() method to generate output predictions based on the learned relationship (Scikit-Learn Developers, 2024a.) We see that the model has internalized $y = 2X + 1$. This reinforces our assumption that the model is effective, as it is able to generalize on data that it hasn't seen before (as long as the data follows the same underlying pattern.)

```
Predicted values = [13. 15. 17. 19.]
```

```
In [578... #Let's add the Boston Housing Dataset as a DataFrame, we can read the CSV directly
using pandas.read_csv()
import pandas as pd
url =
"https://raw.githubusercontent.com/selva86/datasets/master/BostonHousing.csv"
df = pd.read_csv(url)
#(Pandas Development Team, 2024)
print(df.head(3))
print(df.tail(3))
#That looks terrible, some of the columns are cut off. Let's use IPython's display
to make it look better (GeeksforGeeks, 2024)
```

	crim	zn	indus	chas	nox	rm	age	dis	rad	tax	ptratio	\
0	0.00632	18.0	2.31	0	0.538	6.575	65.2	4.0900	1	296	15.3	
1	0.02731	0.0	7.07	0	0.469	6.421	78.9	4.9671	2	242	17.8	
2	0.02729	0.0	7.07	0	0.469	7.185	61.1	4.9671	2	242	17.8	

	b	lstat	medv
0	396.90	4.98	24.0
1	396.90	9.14	21.6
2	392.83	4.03	34.7

	crim	zn	indus	chas	nox	rm	age	dis	rad	tax	ptratio	\
503	0.06076	0.0	11.93	0	0.573	6.976	91.0	2.1675	1	273	21.0	
504	0.10959	0.0	11.93	0	0.573	6.794	89.3	2.3889	1	273	21.0	
505	0.04741	0.0	11.93	0	0.573	6.030	80.8	2.5050	1	273	21.0	

	b	lstat	medv
503	396.90	5.64	23.9
504	393.45	6.48	22.0
505	396.90	7.88	11.9

In [579...

```
from IPython.display import display
#(GeeksforGeeks, 2024)
display(df.head(3))
display(df.tail(3))
#Much better
```

#With the cleaner table output, we can now start to familiarize ourselves with the dataset. We observe how most of the features in the dataset are continuous numerical values, and variables like crim, rm, lstat, and medv span a wide range of real numbers rather than discrete categories or boolean true/false. This means that our dataset seems to be well suited for some type of regression. LR assumes a continuous, linear, and (somewhat) normal distribution which we will confirm in lower cells. Additionally, just from the first and last 3 rows we can already spot some variance in scale (i.e. tax values in the hundreds, vs. crim values which are small decimals < 1. This wide variance in scale is something to keep in mind, as it can influence the learning process and coefficient magnitudes in our LR model. Although LinearRegression in sklearn doesn't explicitly require feature scaling, understanding how scale could affect interpretation will become important in later analysis. For now our initial inspection confirms that the data appears numeric, continuous, and structurally suitable for LR modeling

	crim	zn	indus	chas	nox	rm	age	dis	rad	tax	ptratio	b	lstat	medv
0	0.00632	18.0	2.31	0	0.538	6.575	65.2	4.0900	1	296	15.3	396.90	4.98	24.0
1	0.02731	0.0	7.07	0	0.469	6.421	78.9	4.9671	2	242	17.8	396.90	9.14	21.6
2	0.02729	0.0	7.07	0	0.469	7.185	61.1	4.9671	2	242	17.8	392.83	4.03	34.7



	crim	zn	indus	chas	nox	rm	age	dis	rad	tax	ptratio	b	lstat	n
503	0.06076	0.0	11.93	0	0.573	6.976	91.0	2.1675	1	273	21.0	396.90	5.64	
504	0.10959	0.0	11.93	0	0.573	6.794	89.3	2.3889	1	273	21.0	393.45	6.48	
505	0.04741	0.0	11.93	0	0.573	6.030	80.8	2.5050	1	273	21.0	396.90	7.88	

In [580...

```
#But what are all of those abbreviations for?
boston_column_info = {
    "crim": "Per capita crime rate by town",
    "zn": "Proportion of residential land zoned for lots over 25,000 sq.ft.",
    "indus": "Proportion of non-retail business acres per town",
    "chas": "Charles River dummy variable (= 1 if tract bounds river; 0
otherwise)",
    "nox": "Nitric oxides concentration (parts per 10 million)",
    "rm": "Average number of rooms per dwelling",
    "age": "Proportion of owner-occupied units built prior to 1940",
    "dis": "Weighted distances to five Boston employment centres",
    "rad": "Index of accessibility to radial highways",
    "tax": "Full-value property-tax rate per $10,000",
    "ptratio": "Pupil-teacher ratio by town",
    "b": "1000(Bk - 0.63)^2 where Bk is the proportion of Black residents by
town",
    "lstat": "% lower status of the population",
    "medv": "Median value of owner-occupied homes in $1000s"
}
 #(University of Toronto, 1996)

# Show all column names with their descriptions
for col in df.columns:
    column_info = boston_column_info.get(col, "No description available")
    print(f"{col:7} = {column_info}")

#To make sense of the abbreviated column names, we've created a dictionary mapping
each feature to a human-readable description. By iterating through df.columns
using a for loop, we can print each column alongside its definition for quick
reference. This helps us better understand the meaning of each variable before
going deeper into analysis or model building
```

```
crim    = Per capita crime rate by town
zn      = Proportion of residential land zoned for lots over 25,000 sq.ft.
indus   = Proportion of non-retail business acres per town
chas    = Charles River dummy variable (= 1 if tract bounds river; 0 otherwise)
nox     = Nitric oxides concentration (parts per 10 million)
rm      = Average number of rooms per dwelling
age     = Proportion of owner-occupied units built prior to 1940
dis     = Weighted distances to five Boston employment centres
rad     = Index of accessibility to radial highways
tax     = Full-value property-tax rate per $10,000
ptratio = Pupil-teacher ratio by town
b       = 1000(Bk - 0.63)^2 where Bk is the proportion of Black residents by town
lstat   = % lower status of the population
medv    = Median value of owner-occupied homes in $1000s
```

In [581...

```
#The variable 'b' here, used here as a predictor, appears to reflect some systemic racial bias from the dataset by Harrison and Rubinfeld in 1978. I don't like it, let's get rid of 'b' as a predictor of housing prices for now, maybe we'll explore later what reintroducing 'b' does to our regression model later
print(df.columns)
df = df.drop(columns='b')
print(df.columns)
```

```
Index(['crim', 'zn', 'indus', 'chas', 'nox', 'rm', 'age', 'dis', 'rad', 'tax',
      'ptratio', 'b', 'lstat', 'medv'],
      dtype='object')
Index(['crim', 'zn', 'indus', 'chas', 'nox', 'rm', 'age', 'dis', 'rad', 'tax',
      'ptratio', 'lstat', 'medv'],
      dtype='object')
```

In [582...

```
#Train/Test split 80/20 Let's see how our model performs without any preprocessing.
from sklearn.model_selection import train_test_split
# Since we're going to use Median value as our target, let's exclude it from the predictors as well

#We'll create our input feature matrix by removing the target variable 'medv' from the original DataFrame. We can't include our target in the feature set, otherwise the model is cheating and of no value
X = df.drop(columns='medv')

#Rather, we'll extract the target variable from the original DataFrame and store it as y
y = df['medv']

#Train/Test split, we are creating here separate training and testing datasets for features X and target y
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2,
    #remove the comment from below to make the split reproducible
    #random_state=13
)

#Print the dimensions of the training and testing data to confirm the split was performed correctly. ".shape" call reveals the number of samples (rows) and features (columns.) By confirming the sizes are correct we can ensure that we have a matching number of feature-target pairs in each set, and that 80%/20% of the original data was correctly allocated.
print("Training features shape:", X_train.shape)
print("Training target shape: ", y_train.shape)
print("Testing features shape: ", X_test.shape)
print("Testing target shape:  ", y_test.shape)

#In this cell, we prepared our dataset for training and evaluation by performing a standard 80%/20% train/test split. We need to remove the target 'medv' column from the DataFrame to form our feature matrix X, ensuring that the target variable isn't included as an input. This would invalidate the model's predictive value. We then declare 'medv' as the target vector y, which contains the values the model is trying to predict. We will print the shapes of the resulting datasets to confirm the split was performed as expected.
```

Training features shape: (404, 12)
 Training target shape: (404,)
 Testing features shape: (102, 12)
 Testing target shape: (102,)

In [583... *#Declare a new variable 'model' and assign it the result of calling the LR constructor*
model = lr()
#Train the LR model using the training feature matrix (X_train) and target vector y_train to learn the relationship between inputs and outputs
model.fit(X_train, y_train)

#This cell contains just two lines of code, but they represent an important step in the ML workflow. First, the 'model' variable is initialized with an instance of sklearn's LinearRegression, which is actually a class object which contains the built in methods .fit(), .predict(), and properties like .coef_ and .intercept_. This process enables the model to learn the underlying linear patterns in the data and forms the foundation for future predictions and evaluations. Although compact, this cell marks the transition from data preparation to model application, and successful execution here means that we likely have a working regression pipeline

Out[583... **LinearRegression** ⓘ ⓘ

LinearRegression()

In [584... *#Print a side-by-side comparison of actual vs predicted median values (in thousands) for the first few rows using IPython's display() function*
y_pred = model.predict(X_test)
comparison_df = pd.DataFrame({'Actual': y_test.values, 'Predicted': y_pred}).round(2)
display(comparison_df.head(10))

#Although this is just a quick overview, we can see that the model's predictions are significantly off from the actual values in some cases, especially row 1, where the predicted value is off by over \$9,000 below the actual median price. While this preview doesn't tell the whole story, it is showing that we've got potential for substantial error. The model seems to be running correctly, but we haven't applied much preprocessing of the dataset. While LR doesn't require feature scaling, we saw earlier the large differences in the magnitudes of inputs and perhaps this is distorting our learned coefficients and affecting the model's sensitivity. LR is sensitive to outliers, and we haven't performed any outlier detection or removal which could also be impacting the fit. Finally, we've assumed a linear relationship between all predictors and the target, but that might not be the case with this data. If any relationships are nonlinear, the model won't effectively capture them.

	Actual	Predicted
0	17.5	18.34
1	18.5	19.24
2	19.5	18.79
3	18.3	20.77
4	10.8	11.82
5	13.8	4.38
6	29.1	30.31
7	18.2	14.59
8	22.3	26.77
9	24.1	25.49

In [585...

```
#Let's run some evaluation metrics to measure the model's overall predictive performance
from sklearn.metrics import mean_squared_error  #(Scikit-Learn Developers, 2024b)
from sklearn.metrics import r2_score  #(Scikit-Learn Developers, 2024c)

#Declare the variable y_pred and assign it the result of model.predict(X_test.)
This will hold the predicted values generated by the LR model for the test set
inputs. So y_pred is what the model's estimate of what the median housing prices
should be, based on the features in X_test
y_pred = model.predict(X_test)

#Evaluate the model's performance by calculating the mean squared error
mse = round(mean_squared_error(y_test, y_pred), 2)
#Evaluate the model's performance by calculating the R2 score
r2 = round(r2_score(y_test, y_pred), 2)

print("Mean Squared Error (MSE):", mse)
print("R2:", r2)

#In this cell we evaluate the model by generating predictions and calculating MSE
and R2. MSE penalizes larger errors more heavily (by squaring them), and is
sensitive to outliers and large deviations. MSE provides a numeric interpretation
of error magnitude, how far off are predictions from the actual price. (EDIT: we
have random_state turned on and as such will get different results than the first
time I went through this.) An MSE of 20.09 shows that on average, the difference
between predicted and actual median home prices (when squared) is approximately
$20,090. To get a more intuitive understanding of the model we can take the Root
Mean Squared Error (RMSE) which ends up being aprox $4,480 on average. Whether
this is good or bad depends on the range and distribution of the target variable.
In this dataset, median home prices range from approximately $5,000 to $50,000
(i.e. 5 to 50), and an RMSE of 4.48 on a scale of 5 to 50 represents about a 10%
error relative to the target range. Not great, not terrible. This level of error
suggests that the model is capturing the general trend in the data, but some
preprocessing as described earlier might be useful. An R2 score of 0.79 indicates
that approximately 79% of the variance in median home prices can be explained by
```


features included in the model. I interpret this as the LR model is capturing a substantial portion of the underlying patterns in the data. While it's not a perfect fit, an R^2 of 0.79 would be considered strong for a real-world regression problem, especially considering we're using untransformed features and a basic LR model. The remaining 21% of the variance could be due to factors not captured by the model like nonlinear relationships or the presence of influential outliers.

Mean Squared Error (MSE): 17.12

R^2 : 0.73

```
In [586... #Let's implement some preprocessing to see if we can get a better model fit by
addressing the wide variance in feature scales using sklearn's StandardScaler
from sklearn.preprocessing import StandardScaler as ss #(Scikit-Learn Developers,
2024d)

#Declare the variable scaler and assign it an instance of the ss class. This
object will have access to the .fit(), .transform(), and .fit_transform() methods
used to normalize our dataset. Calling ss() will instantiate a new scaler object
scaler = ss()

#Calculate the mean and standard deviation ( $\sigma$ ) for each feature in X, then
transform the data so that each feature has a mean of zero and a  $\sigma$  of 1
X = scaler.fit_transform(X)

#We'll also need to re-run train/test split again after scaling X (This one took
me a while to figure out why my results weren't changing!!)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2,
    #remove the comment from below to make the split reproducible
    #random_state=13
)

#Since we've just applied ss to X, we'll need to reassign model = lr() before
calling .fit() so that we can start with a fresh model trained on the scaled data,
not one still fitted to unscaled features. This line is not redundant, it's
required because the previous model instance was fitted on unnormalized input and
wouldn't reflect the transformations we've just applied
model = lr()
#Again we'll need to train the lr model using the newly scaled features (X_train)
and their corresponding target values (y_train)
model.fit(X_train, y_train)

#Generate predicted values for the test set by applying the lr model to the newly
scaled test features
y_pred = model.predict(X_test)

#This cell prepares the model to test whether scaling the input features improves
our performance metrics. After normalizing X using StandardScaler so that each
feature has a mean of 0 and a  $\sigma$  of 1, we reinitialize and retrain the lr model to
ensure it's fitted to the scaled data, and then generate new predictions using the
scaled test set. We're trying to evaluate whether applying this preprocessing
method will yield better evaluation metrics, and will run the MSE and  $R^2$  test
again in the next cell
```

```
In [587... #This is just a copy/paste of the previous cell,
```



```

y_pred = model.predict(X_test)

mse = round(mean_squared_error(y_test, y_pred), 2)
r2 = round(r2_score(y_test, y_pred), 2)

print("Mean Squared Error (MSE):", mse)
print("R²:", r2)

#We repeat the same model evaluation process after applying some preprocessing and
reordering the workflow. By running the .predict() method on the test set and
computing MSE and R² we can see that we've achieved a noticeable improvement in
model performance. After scaling the input features and reinitializing the model,
the MSE dropped and the R² rose, indicating a better alignment between the
predicted and actual values. This suggests that reordering the workflow to include
normalization before model fitting helps the model converge more effectively and
makes the learning process more stable and consistent, especially when input
features differ significantly in scale. The decrease in MSE means our predictions
(on average) are closer to the actual values, and the increase in R² shows that a
greater portion of the variance in housing prices are now understood by the model.
I am happy enough with these scores, and we can move on to some data visualization
next.

```

Mean Squared Error (MSE): 14.72
R²: 0.74

In [622...

```

#Let's run the model many times and check out the distribution of our model's
metrics
import numpy as np

#We'll start by making an empty list to store the output of each run
mse_list = []
r2_list = []

#Run the model 100 times in a for loop just like before
for i in range(100000): #Feel free to increase the iterations to explore the law
of large numbers
    #Train/test split, same as before
    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.2,
        #remove the comment from below to make the split reproducible
        #random_state=13
    )

    #Create a new instance of the model for each iteration
    model = lr()
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)

    #Calculate MSE and R² same as before
    mse_list.append(mean_squared_error(y_test, y_pred))
    r2_list.append(r2_score(y_test, y_pred))

#Convert the now populated lists of scores into NumPy arrays for analysis and
plotting
mse_array = np.array(mse_list)
r2_array = np.array(r2_list)

```

#Create a side-by-side histogram showing MSE and R^2 results. I used an LLM to generate this code for me (Grimoire, 2025)

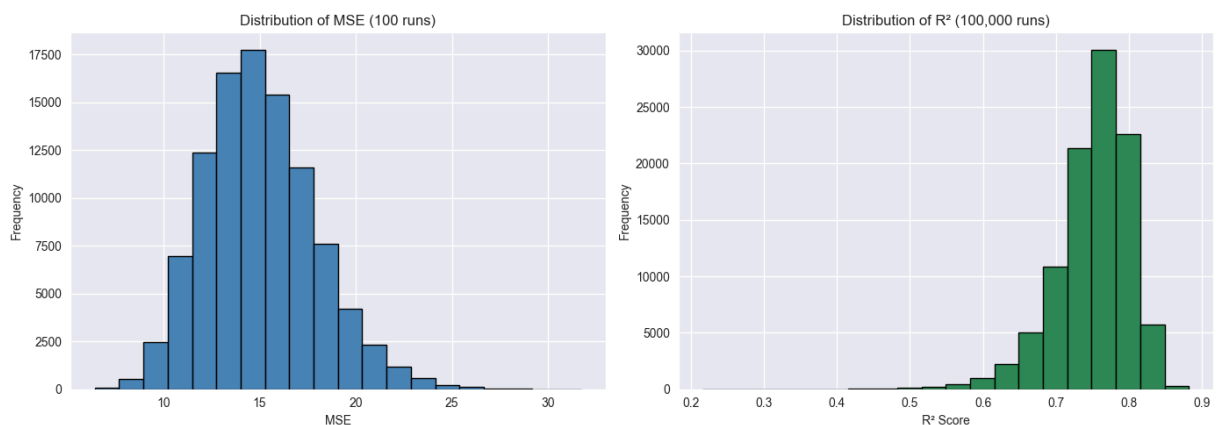
```
fig, axes = matplotlib.subplots(1, 2, figsize=(14, 5))

axes[0].hist(mse_array, bins=20, color='steelblue', edgecolor='black')
axes[0].set_title("Distribution of MSE (100 runs)")
axes[0].set_xlabel("MSE")
axes[0].set_ylabel("Frequency")

axes[1].hist(r2_array, bins=20, color='seagreen', edgecolor='black')
axes[1].set_title("Distribution of  $R^2$  (100,000 runs)")
axes[1].set_xlabel(" $R^2$  Score")
axes[1].set_ylabel("Frequency")

matplotlib.tight_layout()
matplotlib.show()
#(Grimoire, 2025)
```

#I got curious to see if increasing the number of iterations in our randomized train/test split would give a more normalized and stable distribution of performance metrics and set the loop to run 100,000 times. The results show that both MSE and R^2 distributions became much smoother and more tightly clustered around their central values. MSE seems to be maintaining a bit of a right skew, which is probably typical when squaring errors, but its peak is well-defined. The R^2 seems nearly symmetric centered around ~ 0.73 , reflecting consistent model performance across a wide range of splits. This shows us that our model is statistically stable, and that variability due to random splitting diminishes significantly with large-scale resampling. With enough iterations, the model's true behavior becomes more clear



In [616...

#Let's make a scatter plot to visually compare the actual housing prices with the prices predicted by the model. I used an LLM to generate this code for me (Grimoire, 2025)

```
import matplotlib.pyplot as matplotlib

# Scatter plot of actual vs. predicted values
matplotlib.figure(figsize=(8, 6))
matplotlib.scatter(y_test, y_pred, color='blue', alpha=0.6)
matplotlib.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], 'r--',
linewidth=2)
matplotlib.xlabel("Actual MEDV")
matplotlib.ylabel("Predicted MEDV")
```

```

matplot.title("Actual vs Predicted Housing Prices")
matplot.grid(True)
matplot.tight_layout()
matplot.show()
#(Grimoire, 2025)

```

#The red dashed line here represents the ideal case where predicted values exactly match actual values. This serves as a visual benchmark for the model's accuracy. The closer the data points lie to this line, the more accurate the model's predictions were. Deviations from the line indicate prediction errors. We can see that we did pretty well as most of the points are clustered around the line, however there are some noticeable outliers which could be contributing to the residual error in our MSE score.



In [617...

#Let's address some of those outliers. The low-hanging fruit here is that the Boston Housing dataset contains values capped at a maximum of 50.0 for median value, which is not really a true observation, but rather an artificial limit. These high-end outliers are distorting our model fit and are likely causing elevated MSE. Let's just drop those from our dataframe and see what happens

```

#Filter the DataFrame to exclude rows where medv == 50, which is an artificial cap
df = df[df['medv'] < 50.0]

```

```

#Again, we'll remove medv from our features (X) because it is the target (y)
X = df.drop(columns='medv')
y = df['medv']

```

#Train/test split again on the truncated dataset

```

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2,
    #remove the comment from below to make the split reproducible
    #random_state=13
)

#
model = lr()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

mse = round(mean_squared_error(y_test, y_pred), 2)
r2 = round(r2_score(y_test, y_pred), 2)

print("Mean Squared Error (MSE):", mse)
print("R²:", r2)

#Looks promising

```

Mean Squared Error (MSE): 12.85
R²: 0.75

In [621...

#Let's repeat our looping experiment, now with the trimmed data. By running the model n times with different random splits of the trimmed data, we can get a better idea of how this preprocessing step will affect MSE and R². Same logic as before.

#Let's change up the variable names to reflect the trimmed dataset

```

mse_trimmed = []
r2_trimmed = []

```

```

for i in range(100000):
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)
    model = lr()
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)
    mse_trimmed.append(mean_squared_error(y_test, y_pred))
    r2_trimmed.append(r2_score(y_test, y_pred))

```

```

mse_trimmed_array = np.array(mse_trimmed)
r2_trimmed_array = np.array(r2_trimmed)

```

```

fig, axes = matplotlib.subplots(1, 2, figsize=(14, 5))

```

```

axes[0].hist(mse_trimmed_array, bins=20, color='steelblue', edgecolor='black')
axes[0].set_title("Distribution of MSE (Trimmed Data, 100 runs)")
axes[0].set_xlabel("MSE")
axes[0].set_ylabel("Frequency")

```

```

axes[1].hist(r2_trimmed_array, bins=20, color='seagreen', edgecolor='black')
axes[1].set_title("Distribution of R² (Trimmed Data, 100,000 runs)")
axes[1].set_xlabel("R² Score")
axes[1].set_ylabel("Frequency")

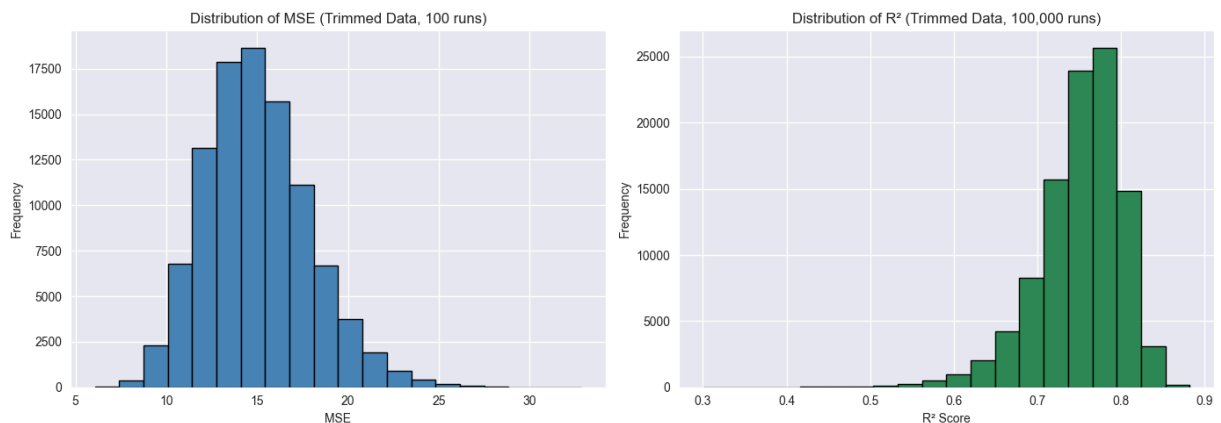
```

```

matplotlib.tight_layout()
matplotlib.show()

```

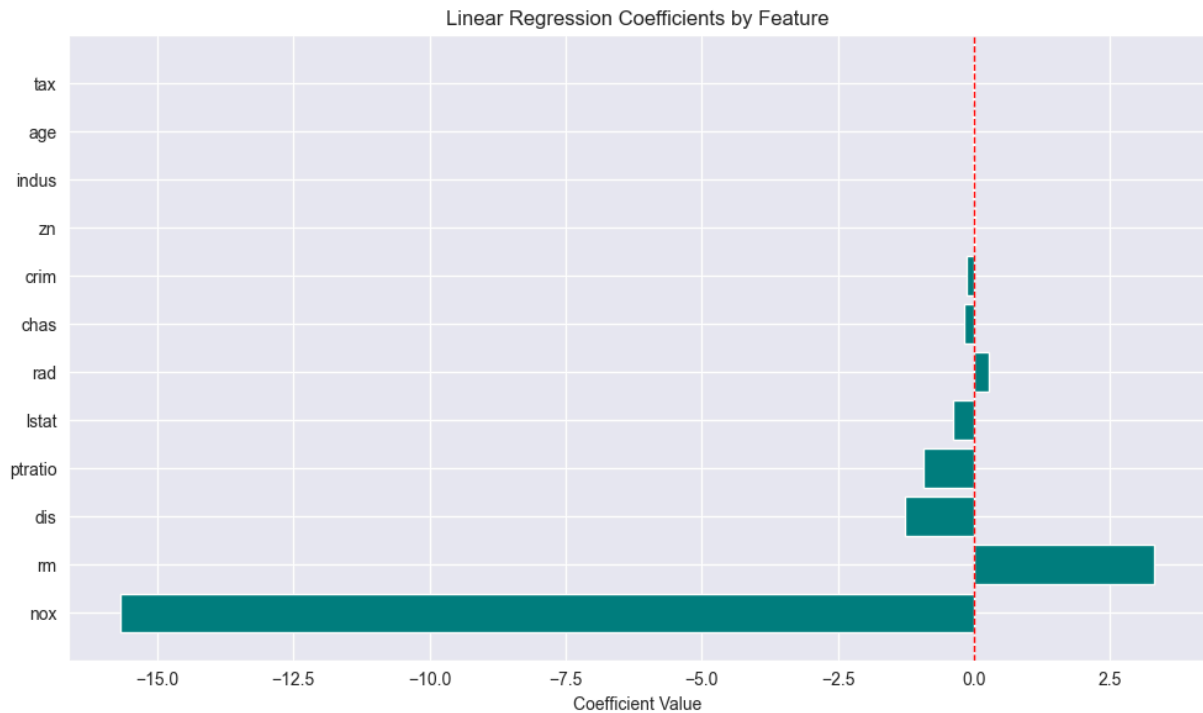
#Running the loop 100,000 times, we can observe that the MSE distribution shifts leftward just a little bit, peaking around 14-15. This suggests a modest reduction in average error and improved consistency though the change is not dramatic. The R^2 scores remain tightly clustered around 0.75-0.80, indicating stable model performance across splits, with only a slight improvement in variance after trimming the capped outliers. Overall, the comparison shows that removing the data when $\text{medv} == 50$ has a subtle but measurable effect. My main takeaway from this experiment is that the law of large numbers can compensate for the presence of moderate outliers when the model is evaluated over a sufficiently large number of randomized train/test splits. Rather than relying on manual data manipulation, repeated resampling allows the model's true performance characteristics to emerge. In this case, running the model 100,000 times smooths out the variability introduced by outliers, showing that the core performance metrics remain stable even without trimming the dataset.



In [592...

#I wanted to see which features had the strongest positive or negative influence on housing prices. I asked the LLM (Grimoire, 2025) to create a horizontal bar chart to visualize which features had the most influential effect on medv . A positive coefficient shows that an increase in that feature tends to raise the predicted medv (in this case it was average number of rooms), and a negative coefficient means the opposite. Pollution (nox = nitric oxide concentration) had the most substantial negative effect, followed by distance to employment centers, and then pupil-teacher ratio, where having a larger distance or ratio of students:teachers tends to have a negative impact on medv .

```
matplotlib.figure(figsize=(10, 6))
matplotlib.barh(coefficients["Feature"], coefficients["Coefficient"], color='teal')
matplotlib.xlabel("Coefficient Value")
matplotlib.title("Linear Regression Coefficients by Feature")
matplotlib.axvline(x=0, color='red', linestyle='--', linewidth=1)
matplotlib.grid(True, axis='x')
matplotlib.tight_layout()
matplotlib.show()
#(Grimoire, 2025)
```



In [635...

```
#I was also curious whether any of the features shared colinearity, as a strong linear relationship between the predictors themselves can undermine the reliability of a linear regression model (Hayes, 2024.) Seaborn (n.d.) has a heatmap function that makes it easy to visualize correlations between features. I had used sns.heatmap before when making confusion matrices. Using the same function to make a colinearity heatmap we can integrate matplotlib's .show() function, since Seaborn relies on Matplotlib as its underlying plotting engine
import seaborn as sns

matplotlib.figure(figsize=(13, 13))

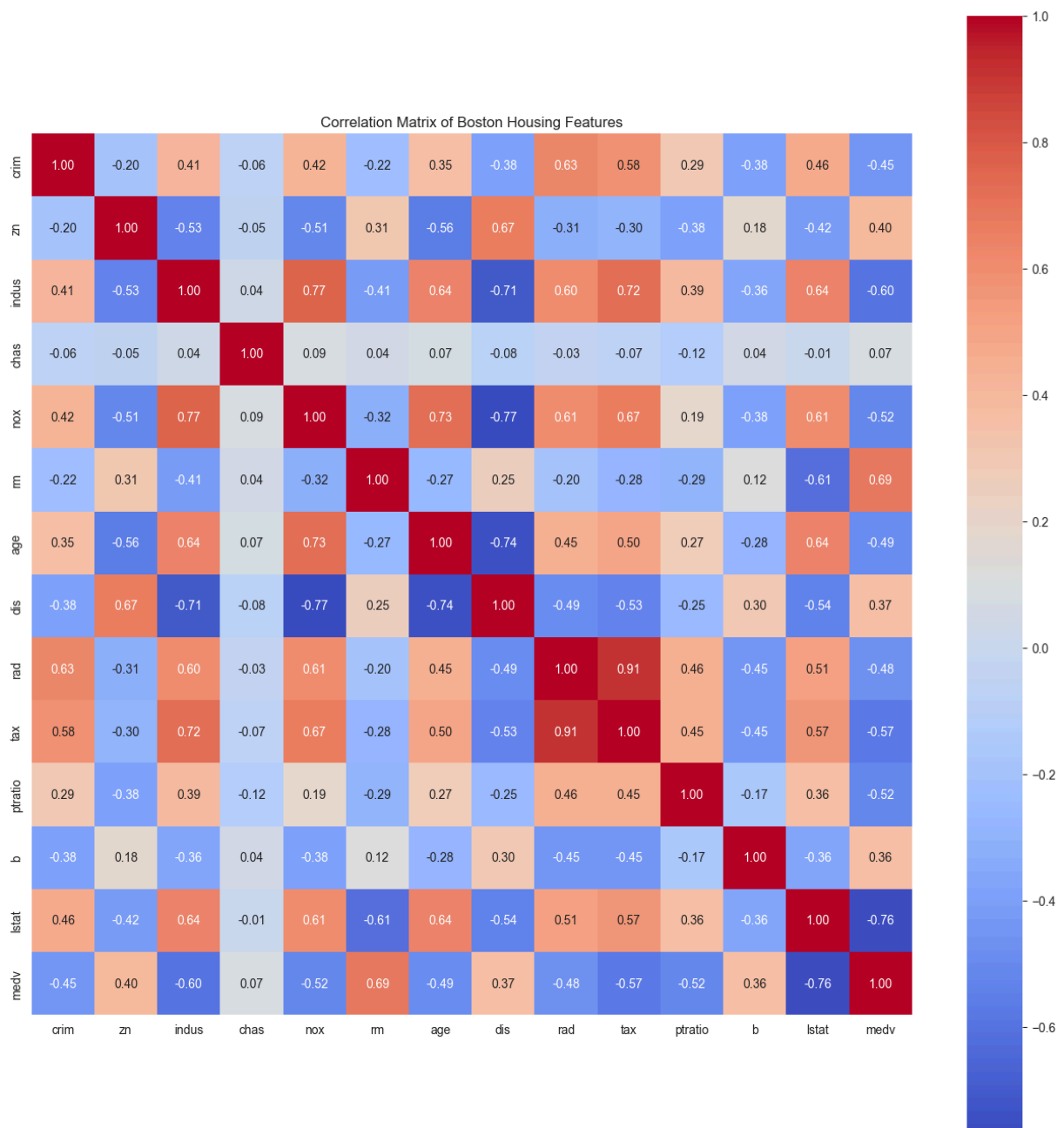
#Using our existing Dataframe, we can call .corr() to generate a matrix of Pearson correlation coefficients, which is appropriate when assessing linear relationships between continuous variables, especially in the case of an LR model (Sereno, 2025.)
correlation_plot = df.corr()

#Using Seaborn's heatmap as a function of our dataframe lets us select what parameters we want to control the appearance and level of detail in the plot. Here, we will leave everything as default except for annot which enables numerical correlation labels, fmt which formats to 2 decimal places, cmap to give a cool/warm color palate appropriate for this plot, square to make uniform cell dimensions, and cbar to show a color legend representing correlation strenght
sns.heatmap(correlation_plot, annot=True, fmt=".2f", cmap="coolwarm", square=True, cbar=True)
matplotlib.title("Correlation Matrix of Boston Housing Features")

#A tight layout works here to ensure all plot elements are neatly arranged within the figure's boundaries
matplotlib.tight_layout()
matplotlib.show()

#This heatmap visualizes the pearson correlation matrix of the Boston Housing
```

DataFrame. As expected it shows a strong correlation in medv with rooms per dwelling, suggesting more spacious interiors tend to draw a higher median price. Looking for colinearity between the independent features of the data, the index of accessibility to highways and the full-value property-tax rate are highly correlated (0.91.) Polution concentration also shows high correlation with the proportion of industrial zoning, which is not surprising. If our goal is to interpret which variables drive median price, it might be worth considering dropping one of each pair of those variables. An interesting statistical technique called Variance Inflation Factor (Penn State University, n.d.) could be used to determine which variable to remove. VIF quantifies how much a variable's variance is inflated due to its correlation with other predictors, and might be helpful in identifying which features are distorting the interpretability of our LR model. Alternatively, if we were using a regularization regression model (i.e. ridge or lasso), we wouldn't have to worry about removing them, as regularization would dampen the coefficients of redundant predictors without the need to manually remove them



In [599...

```

#GeeksforGeeks. (2024). Display vs print in Pandas.
#https://www.geeksforgeeks.org/display-vs-print-in-pandas/

#GeeksforGeeks. (2025). Interquartile Range to Detect Outliers in Data.
#https://www.geeksforgeeks.org/interquartile-range-to-detect-outliers-in-data/

#Grimoire. (2025). Conversation about data visualization in a linear regression
model. OpenAI.
#https://chat.openai.com/

#Hayes, A. (2024). Multicollinearity: Meaning, examples, and FAQs. Investopedia.
#https://www.investopedia.com/terms/m/multicollinearity.asp

#Mahmood, M. (2024, March 24). The meaning of capital X and small y in machine
Learning. Medium.
#https://medium.com/@mohamad.razzi.my/the-meaning-of-capital-x-and-small-y-in-
machine-learning-c7f8ac5ffd38

#Pandas Development Team. (2024). pandas.read_csv (Version 2.2.2) Pandas.
#https://pandas.pydata.org/docs/reference/api/pandas.read_csv.html

#Penn State University. (n.d.). 10.7 - Detecting multicollinearity using variance
inflation factors. STAT 462: Applied Regression Analysis.
#https://online.stat.psu.edu/stat462/node/180/

#University of Toronto. (1996). The Boston Housing Dataset. DELVE: Data for
Evaluating Learning in Valid Experiments.
https://www.cs.toronto.edu/~delve/data/boston/bostonDetail.html

#Scikit-Learn Developers. (2024a). sklearn.linear_model.LinearRegression. Scikit-
Learn.
#https://scikit-
Learn.org/stable/modules/generated/sklearn.linear_model.LinearRegression.html

#Scikit-Learn Developers. (2024b). sklearn.metrics.mean_squared_error. Scikit-
Learn.
#https://scikit-
Learn.org/stable/modules/generated/sklearn.metrics.mean_squared_error.html

#Scikit-Learn Developers. (2024c). sklearn.metrics.r2_score. Scikit-Learn.
#https://scikit-learn.org/stable/modules/generated/sklearn.metrics.r2_score.html

#Scikit-Learn Developers. (2024d). sklearn.preprocessing.StandardScaler. Scikit-
Learn.
#https://scikit-
Learn.org/stable/modules/generated/sklearn.preprocessing.StandardScaler.html

#Seaborn. (n.d.). seaborn.heatmap. Seaborn Documentation.
#https://seaborn.pydata.org/generated/seaborn.heatmap.html

#Sereno. (2025). Comparison of Pearson vs Spearman correlation coefficients.
Analytics Vidhya.
#https://www.analyticsvidhya.com/blog/2021/03/comparison-of-pearson-and-spearman-
correlation-coefficients/

```